

ComicCraft: AI Comic Story Creator using Gemini Models

Project Documentation & Implementation Report

Team Leader: Dharshini.R

E-mail- dharsniniravi81@gmail.com

NM id- A0D580E10C608DE862340BC15848BC32

Team Members:

1. Nithisha.S

E-mail- nithishasara9@gmail.com

NM id- A47618E38C9D78AA0314DC4E8BD4BD32

2. Indumathi.B

E-mail- indubabu2008@gmail.com

NM id- D477BA4295D2327BCFC48A83CE5DE140

3. Swathy.

E-mail- swathydevaraj3001@gmail.com

NM id-18CABB3F3268C8A9F084B159F0D7AB4F

4. Yogalakshmi.

E-mail- yogalakshmiyoga153@gmail.Com

NM id- 843EA74A32FDF828F7C4C86DD044E320

Department: Bsc Computer Science with Data Science

Institution:

Academic Year: 2025–2028

Table of Contents

1. Phase 1: Brainstorming & Ideation Phase
2. Phase 2: Requirement Analysis Phase 3.
- Phase 3: Project Design Phase
4. Phase 4: Project Planning Phase 5.

Phase 5: Project Development Phase 6.
Phase 6: Project Testing Phase 7. Phase
7: Project Documentation Phase 8. Phase
8: Project Demonstration Phase 9.
Conclusion & Future Enhancements

PHASE 1: Brainstorming & Ideation Phase

1.1 Background and Motivation

The proliferation of generative artificial intelligence over the past several years has fundamentally altered the manner in which creative content is conceived, produced, and consumed. Large Language Models (LLMs) such as Google's Gemini family, alongside diffusion-based image synthesis models such as Stable Diffusion, have democratized access to creative tooling that was previously the exclusive domain of trained illustrators, scriptwriters, and graphic designers. Comic books and sequential visual storytelling have historically required a multidisciplinary production pipeline involving a writer to draft the narrative, a penciller to sketch the panels, an inker and colorist to finalize the artwork, and a letterer to add dialogue and narration. This division of labor, while producing high-quality creative output, imposes a substantial barrier to entry for hobbyists, students, educators, and independent creators who wish to visualize a story idea but lack either the artistic skill or the financial resources to commission a professional illustrator.

The ComicCraft project was conceived as a direct response to this gap. The central motivating question guiding the ideation phase was: *"Can an end-to-end automated pipeline, orchestrated entirely through natural language prompts, produce a coherent, visually consistent, multi-panel comic strip without requiring the end user to possess any artistic or technical skill?"* This question naturally decomposes into two sub-problems that map directly onto the two dominant categories of generative AI available today — narrative generation (a natural language processing problem, well-suited to LLMs) and visual generation (an image synthesis problem, well-suited to diffusion models). The brainstorming phase therefore focused heavily on identifying how these two categories of models could be bridged programmatically, and what intermediate data structures (i.e., structured JSON-like panel outlines) would be required to pass information from the narrative-generation stage to the image-generation stage without human intervention.

1.2 Ideation Workshops and Concept Sketching

During the initial ideation sessions, the team conducted structured brainstorming exercises, including mind-mapping and "How Might We" (HMW) statement generation, to explore the solution space before committing to an architecture. Several HMW statements were drafted and evaluated:

- **How might we allow a user with zero drawing ability to produce a shareable comic strip in under two minutes?**
- **How might we ensure narrative coherence across multiple panels when each panel's artwork is generated independently by a stateless image model?**
- **How might we allow customization of tone, setting, and art style without overwhelming the user with technical configuration options?**
- **How might we package the final output into a portable, distributable format (PDF) suitable for sharing on social media or printing?**

These questions collectively shaped the scope of the project. The team ultimately converged on a five-panel comic strip as the target output granularity, as this was judged to be long enough to convey a complete narrative arc (introduction, rising action, climax, and resolution can be compressed into five beats) while remaining short enough to keep image-generation latency and API costs manageable within the constraints of a student project running on Hugging Face's open-source Stable Diffusion v1.5 checkpoint on local/CPU-bound hardware.

1.3 Problem Statement

Manual comic creation is a time-intensive, multi-skill process that is inaccessible to individuals without artistic training, illustration software proficiency, or a budget to commission freelance artists. Existing AI-assisted creative tools tend to address only one half of the problem — either they generate a story (text-only chatbots) or they generate isolated images (single-prompt image generators) — but very few consumer-accessible tools chain these two modalities together into a single, coherent, multi-panel sequential-art pipeline with a user-friendly web interface and exportable output. ComicCraft addresses this

gap by providing a unified web application in which a single natural-language story prompt, combined with a handful of structured metadata fields (character name, setting, tone, art style), is automatically expanded into a five-panel illustrated comic strip complete with narration text, and packaged into a downloadable PDF.

1.4 Target Audience and Use-Case Scenarios

The ideation phase identified four primary user personas whose needs the application should satisfy:

Persona 1 — The Hobbyist Writer: An individual who enjoys writing short-form fiction but has no illustration skills. This user wants to visualize their story ideas quickly to share with friends or on social platforms such as Instagram or Reddit.

Persona 2 — The Educator: A schoolteacher who wishes to generate simple illustrated stories to use as teaching aids for language arts, history, or moral-science lessons, where a visual narrative increases student engagement over plain text.

Persona 3 — The Indie Game/App Developer: A developer prototyping narrative content (e.g., cutscene storyboards, lore documents) who needs rapid visual mockups before commissioning final art from a professional illustrator.

Persona 4 — The Casual Entertainment Seeker: A user who simply wants to experiment with "what if" story prompts (e.g., "an astronaut discovers a glowing alien temple") purely for entertainment and curiosity, with no downstream professional use case.

Each of these personas informed specific design decisions later in the Requirement Analysis and Design phases — for instance, the decision to keep the input form to five fields (prompt, character name, setting, tone, art style) rather than a fully configurable panel-by-panel editor was made explicitly to serve Persona 1 and Persona 4, who prioritize speed and simplicity over granular creative control.

1.5 Feasibility and Scope Boundaries

During ideation, the team also performed an early feasibility assessment to bound the scope of the Minimum Viable Product (MVP). It was recognized early that maintaining perfect character visual consistency across independently generated diffusion panels is an open research problem (addressed by techniques such as IP-Adapter, DreamBooth fine-tuning, or ControlNet reference conditioning), and that implementing such consistency mechanisms was outside the time budget of the project. The team therefore consciously scoped the MVP to accept minor visual inconsistency between panels (e.g., the protagonist's suit color or face may vary slightly panel-to-panel) as an acknowledged limitation, to be listed explicitly in the Future Scope section, rather than attempting to solve it within the current release. This kind of scope-boundary decision-making is a key deliverable of a mature ideation phase and is documented here to demonstrate engineering judgment

PHASE 2: Requirement Analysis Phase

2.1 Overview

Following ideation, the team conducted a formal requirement analysis to translate the conceptual vision into a concrete, testable specification. Requirements are categorized below into Functional Requirements (FR), Non-Functional Requirements (NFR), and Technical/Model Selection Requirements, following standard software engineering documentation conventions (e.g., IEEE 830 style requirement tagging).

2.2 Functional Requirements

ID	Requirement Description
FR-01	The system shall provide a web form allowing the user to input a free-text story prompt.
FR-02	The system shall allow the user to specify a main character name as a string field.
FR-03	The system shall provide a dropdown for selecting the story setting (e.g., Space, Fantasy Kingdom, Urban City, Post-Apocalyptic).
FR-04	The system shall provide a dropdown for selecting the narrative tone (e.g., Dramatic, Comedic, Mysterious, Adventurous).
FR-05	The system shall provide a dropdown for selecting the visual art style (e.g., Comic Book, Manga, Watercolor, Noir).

| FR-06 | Upon submission, the system shall generate a five-panel story outline using Gemini 1.5 Flash. ||

FR-07 | The system shall expand the outline into detailed panel-level narration and dialogue using Gemini 1.5 Pro. |

| FR-08 | The system shall generate one illustrated image per panel using a locally hosted Stable Diffusion v1.5 pipeline, using the selected art style as a stylistic modifier in the prompt. |

| FR-09 | The system shall display a live preview of the generated comic (all five panels with narration overlaid or captioned) on a dedicated preview page. |

| FR-10 | The system shall allow the user to export/download the finished comic as a multi-page PDF file using FPDF. |

| FR-11 | The system shall persist generated panel images to a static directory so they can be re-rendered without regeneration on subsequent page loads within the same session. |

| FR-12 | The system shall display a confirmation/success page after a successful PDF export, including a download link. |

2.3 Non-Functional Requirements

| ID | Requirement Description |

|---|-----|

| NFR-01 | ****Usability:**** The input form shall be completable by a first-time user in under 60 seconds without external instructions. |

| NFR-02 | ****Performance:**** End-to-end comic generation (outline + narration + 5 images) should complete within an acceptable wait time, with progress feedback where generation exceeds 10 seconds. ||

NFR-03 | ****Reliability:**** The system shall gracefully retry failed Gemini API calls (e.g., transient 503/404 errors) up to three times with exponential backoff before surfacing an error to the user. || NFR-04 | ****Portability:**** The application shall run cross-platform (Windows/Linux/macOS) inside a Python virtual environment with pinned dependency versions. |

| NFR-05 | ****Security:**** API keys (Gemini, Hugging Face) shall never be hard-coded into source files; they shall be loaded exclusively from an untracked `.env` file excluded via `.gitignore`. || NFR-06 |

****Maintainability:**** The codebase shall follow a modular separation of concerns (routing, AI integration, image generation, layout/export) to allow independent modification of each pipeline stage. || NFR-07 |

****Scalability (Future):**** While the MVP runs Stable Diffusion on CPU for local/offline development, the

architecture shall not preclude a future migration to GPU-backed or hosted inference endpoints. |

2.4 Model Selection Rationale

A critical requirement-analysis activity was the deliberate, justified selection of AI models for each pipeline stage, rather than an arbitrary choice. The rationale for each selection is documented below to demonstrate engineering due diligence.

2.4.1 Gemini 1.5 Flash for Outline Generation

Gemini 1.5 Flash was selected for the initial story-outline generation stage because outline generation is a comparatively lightweight NLP task — decomposing a single-sentence prompt into five sequential story beats does not require the deepest level of reasoning, but does benefit from low latency, since the outline is the first user-facing step in the pipeline and directly affects perceived responsiveness. Flash-tier models are optimized by Google for high throughput and reduced response time at a lower computational cost than Pro-tier models, making them the appropriate choice for this "fan-out" stage of the pipeline where speed matters more than maximal narrative depth.

2.4.2 Gemini 1.5 Pro for Narration and Dialogue Expansion

Gemini 1.5 Pro was selected for the subsequent narration/dialogue-expansion stage because this stage requires higher-quality creative writing — converting a terse outline bullet point (e.g., "hero discovers portal") into an engaging, comic-book-style narration caption with proper tone, pacing, and (optionally) character dialogue requires stronger long-form language generation capability. Since this stage runs only once per comic (rather than once per panel), the additional latency and computational cost of the Pro-tier model is an acceptable trade-off for improved narrative quality.

2.4.3 Stable Diffusion v1.5 (via Hugging Face Diffusers) for Image Generation

Stable Diffusion v1.5 was selected as the image-generation backbone primarily due to its open-source availability, its mature integration with the Hugging Face `diffusers` Python library, and its ability to run entirely offline/locally without incurring per-image API costs, which was an important consideration for a

student project operating under budget constraints. While newer diffusion checkpoints (e.g., SDXL, SD 3) offer higher fidelity, v1.5 was chosen for its lower VRAM/CPU memory footprint, faster inference on constrained development hardware, and the abundance of community documentation available for troubleshooting.

2.4.4 FPDF for PDF Export

FPDF was selected over heavier alternatives (such as ReportLab or WeasyPrint) because the export requirement is comparatively simple — arranging five images with accompanying caption text into a linear multi-page document — and FPDF's minimal dependency footprint and straightforward procedural API (``add_page()``, ``image()``, ``multi_cell()``) were judged sufficient without introducing unnecessary complexity into the ``exporters.py`` module.

2.5 Data Requirements

The requirement analysis phase also defined the structured data contract passed between pipeline stages, since Gemini's free-text output must be reliably parsed into a machine-readable structure before it can be consumed by the image generator. The team specified that the outline-generation stage (Gemini 1.5 Flash) must return a JSON-formatted list of five objects, each containing ``panel_number``, ``scene_description``, and ``image_prompt`` keys, and that the narration-expansion stage (Gemini 1.5 Pro) must return a parallel structure adding a ``narration_text`` key per panel. This explicit schema requirement, documented during requirement analysis rather than discovered ad hoc during coding, significantly reduced integration bugs during the Development phase.

PHASE 3: Project Design Phase

3.1 High-Level System Architecture

ComicCraft follows a modular, layered architecture that cleanly separates presentation, orchestration, and AI-integration concerns. At the highest level, the system is composed of three cooperating layers:

1. ****Presentation Layer**** — Jinja2-templated HTML pages (``index.html``, ``comic_preview.html``, ``export_success.html``) served by FastAPI's ``Jinja2Templates`` engine, styled with CSS for a clean, form-driven user experience.
2. ****Application/Orchestration Layer**** — FastAPI route handlers (``routes.py``) that receive form submissions, sequentially invoke the outline generator, narration generator, and image generator, assemble the resulting panel objects, and render the preview template.
3. ****AI Integration Layer**** — Discrete Python modules (``gemini_flash.py``, ``gemini_pro.py``, ``image_generator.py``) that each wrap a single external or local model behind a clean function-level interface (``generate_outline()``, ``generate_story()``, ``generate_image()``), allowing each model to be swapped or upgraded independently without modifying calling code elsewhere in the system.

This layered separation follows the well-established Single Responsibility Principle from software engineering best practice: each module has exactly one reason to change. If Google releases a new Gemini model version, only ``gemini_flash.py``/``gemini_pro.py`` require modification; if the team later migrates from Stable Diffusion v1.5 to SDXL, only ``image_generator.py`` requires modification.

3.2 Backend Design (FastAPI)

FastAPI was selected as the backend framework due to its native asynchronous request handling, automatic OpenAPI/Swagger documentation generation, and first-class Pydantic model integration for request validation. The backend exposes a small number of routes:

- ``GET /`` — renders the input form (``index.html``).
- ``POST /generate`` — accepts the ``PromptRequest`` Pydantic model (containing ``prompt``, ``character_name``, ``setting``, ``tone``, ``style``), triggers the full generation pipeline, and renders ``comic_preview.html`` with the resulting panel data.
- ``POST /export`` — invokes ``save_pdf()`` from ``exporters.py`` on the already-generated panel set and renders ``export_success.html`` with a download link to the resulting PDF in ``static/exports/``.

Static assets (generated panel images, exported PDFs, and custom fonts for lettering) are served via FastAPI's ``StaticFiles`` mount at the ``/static`` path, mirroring the directory-structure design established in the

reference project documentation.

3.3 Frontend Design

The frontend was deliberately kept lightweight — plain HTML5 with hand-written CSS rather than a heavier JavaScript framework such as React — since the application's interactivity requirements are modest (a single form submission and two subsequent read-only preview/confirmation pages). The `index.html` template presents a single-column form with the five required input fields (Story Prompt as a `<textarea>`, Character Name as a text `<input>`, and Setting/Tone/Style as `<select>` dropdowns), followed by a prominent "Generate Comic" call-to-action button. The `comic_preview.html` template iterates over the five generated panels, displaying each panel's image alongside its narration caption in a vertically stacked, scrapbook-style layout reminiscent of traditional print comics, followed by an "Export as PDF" button. The `export_success.html` template confirms successful export and surfaces a direct download hyperlink.

3.4 AI Pipeline Design (Sequence Flow)

The following sequence describes the end-to-end data flow through the AI pipeline, which is the technical heart of the system design:

1. User submits the form → FastAPI parses it into a `PromptRequest` object.
2. `routes.py` calls `generate_outline(prompt, character_name, setting, tone)` in `gemini_flash.py`, which returns a five-item structured outline.
3. `routes.py` passes this outline into `generate_story(outline)` in `gemini_pro.py`, which returns enriched narration/dialogue text per panel.
4. For each of the five panels, `routes.py` calls `generate_image(image_prompt, style)` in `image_generator.py`, which invokes the local Stable Diffusion pipeline and writes a PNG file into `static/panels/`.
5. `routes.py` assembles a list of panel dictionaries (`{image_path, narration_text}`) and passes this list into the Jinja2 rendering context for `comic_preview.html`.
6. On export, `save_pdf(panels)` in `exporters.py` iterates the same panel list, drawing each image and its caption onto a new PDF page via FPDF, and writes the final file to `static/exports/`.

3.5 Database and State Design

Given the scope of the MVP, the team made a deliberate design decision to avoid introducing a persistent relational database (e.g., PostgreSQL/SQLite) in the first release. Session state (the current comic's panel data) is held transiently in server memory / passed directly through the render context between the `/generate` and `/export` requests, with generated assets persisted only as files on disk under `static/panels/` and `static/exports/`. This was judged an acceptable trade-off for a single-session, single-user demonstration application, with the Future Scope section explicitly noting database-backed comic history/galleries as a planned enhancement.

3.6 UI/UX Wireframe Design

Prior to implementation, the team sketched low-fidelity wireframes for each of the three screens to validate layout decisions before writing any HTML/CSS. The wireframing exercise confirmed the decision to place all five input fields above the fold on a single screen (rather than a multi-step wizard), in order to satisfy NFR-01 (sub-60-second form completion).

PHASE 4: Project Planning Phase

4.1 Development Methodology

The team adopted a lightweight Agile-inspired iterative methodology, organizing work into short, milestone-driven sprints rather than a rigid Waterfall sequence, given the exploratory nature of integrating multiple unfamiliar AI APIs. Each milestone below concluded with a working, demonstrable increment of functionality, allowing early detection of integration issues (particularly around Gemini API response parsing and Stable Diffusion inference latency) rather than deferring all integration risk to the end of the project timeline.

4.2 Milestone Breakdown

Milestone	Deliverable	Approx. Duration
M1	Environment setup: virtual environment (<code>`comiccra</code>	Week 1
M2	Gemini 1.5 Flash integration: implement and unit-test <code>`generate_outline()</code> in isolation using hard-coded sample prompts.	Week 2
M3	Gemini 1.5 Pro integration: implement and unit-test <code>`generate_story()</code> , validating narration quality against sample outlines from M2.	Week 2–3
M4	Stable Diffusion integration: implement <code>`image_generator.py`</code> , validate local CPU inference, tune prompt-engineering templates per art style.	Week 3–4
M5	Frontend scaffolding: build <code>`index.html`</code> , wire up the <code>`/generate`</code> route, confirm end-to-end pipeline produces five panels.	Week 4–5
M6	Preview page: build <code>`comic_preview.html`</code> , display generated panels with narration captions.	Week 5
M7	PDF export: implement <code>`exporters.py`</code> using FPDF, build <code>`export_success.html`</code> , wire up the <code>`/export`</code> route.	Week 6
M8	Testing & bug-fixing: unit tests, endpoint tests, manual QA across all art-style/	Week 7
M9	Documentation & final demonstration preparation.	Week 8

4.3 Task Allocation and Team Roles

To ensure parallelizable progress and clear accountability, responsibilities were distributed across the team according to individual strengths and the modular architecture defined in Phase 3:

- **Team Leader:** Overall architecture ownership, Gemini API integration (``gemini_flash.py``, ``gemini_pro.py``), and prompt-engineering design for narrative generation.
- **Member 2:** Stable Diffusion integration (``image_generator.py``), art-style prompt template design,

and image-pipeline performance tuning.

- **Member 3:** FastAPI routing (`routes.py`, `main.py`), frontend template development (`index.html`, `comic_preview.html`, `export_success.html`), and CSS styling.
- **Member 4:** PDF export logic (`exporters.py`), testing (unit and endpoint), and documentation compilation.

4.4 Risk Assessment and Mitigation Plan

A formal risk register was maintained throughout planning, identifying the following key risks:

Risk	Likelihood	Impact	Mitigation Strategy
Gemini API rate limits or transient outages (503/404 errors) during demo	Medium	High	Implemented retry-with-backoff logic (<code>MODELS_TO_TRY</code> fallback list, three retry attempts per model) in <code>gemini_pro.py</code> .
CPU-based Stable Diffusion inference too slow for live demo	High	Medium	Pre-generate a cached "known good" demo prompt set as a fallback before the live presentation; document expected latency honestly in the report.
Inconsistent JSON parsing from Gemini free-text responses	Medium	High	Enforced strict prompt instructions requesting JSON-only output; added defensive parsing with fallback error messages.
Scope creep (e.g., attempting speech-bubble rendering or multi-page chapters in the MVP)	Medium	Medium	Explicitly deferred to Future Scope section (see Phase 9) rather than expanding MVP requirements mid-sprint.

4.5 Gantt Chart Overview

PHASE 5: Project Development Phase

5.1 Directory Structure

Following the architecture defined in Phase 3, the project's final directory layout is organized as follows, mirroring a standard modular FastAPI project layout:

...

ComicCraft/

- |— app/
 - | |— __init__.py
 - | |— main.py # FastAPI app initialization & static mounts
 - | |— routes.py # Route definitions & pipeline orchestration
 - | |— gemini_flash.py # Outline generation (Gemini 1.5 Flash)
 - | |— gemini_pro.py # Narration/dialogue generation (Gemini 1.5 Pro)
 - | |— image_generator.py # Stable Diffusion v1.5 image rendering
 - | |— exporters.py # FPDF-based PDF export logic
- |— comiccraft-env/ # Python virtual environment (not version-controlled)
- |— static/
 - | |— panels/ # Generated panel PNG images
 - | |— exports/ # Generated PDF exports
 - | |— fonts/ # Custom lettering fonts (future speech-bubble use)
- |— templates/
 - | |— index.html # Story input form
 - | |— comic_preview.html # Generated comic preview
 - | |— export_success.html # PDF export confirmation page
- |— .env # API keys (GEMINI_API_KEY, HF_API_KEY) — untracked |—
- |— .gitignore
- |— requirements.txt

...

```
app > main.py
1  import os
2  from dotenv import load_dotenv
3
4  # Load .env file before importing routes
5  load_dotenv()
6
7  from fastapi import FastAPI
8  from fastapi.staticfiles import StaticFiles
9  from app.routes import router
10
11 app = FastAPI(title="ComicCraft AI")
12
13 # Ensure required directories exist
14 os.makedirs("static/panels", exist_ok=True)
15 os.makedirs("static/exports", exist_ok=True)
16 os.makedirs("static/fonts", exist_ok=True)
17
18 app.mount("/static", StaticFiles(directory="static"), name="static")
19 app.include_router(router)
```

5.2 Environment Configuration

Sensitive credentials are isolated in an untracked `.env` file, loaded via `python-dotenv` at application startup before any dependent module is imported, ensuring that `os.getenv()` calls inside `gemini_flash.py` and `gemini_pro.py` resolve correctly at import time.

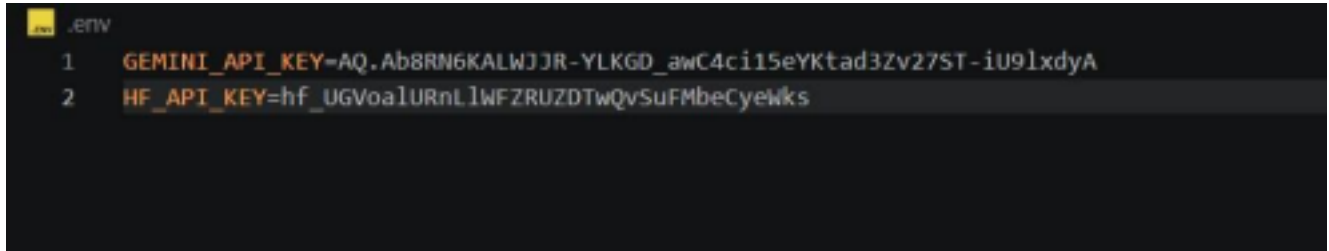
...

.env

GEMINI_API_KEY=your_gemini_api_key_here

HF_API_KEY=your_huggingface_api_key_here

...

A screenshot of a terminal window with a dark background. At the top left, there is a yellow icon and the text '.env'. Below it, two lines of text are displayed: '1 GEMINI_API_KEY=AQ.Ab8RN6KALWJJR-YLKGD_awC4ci15eYKtad3Zv27ST-iU9lxdyA' and '2 HF_API_KEY=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'.

```
.env
1 GEMINI_API_KEY=AQ.Ab8RN6KALWJJR-YLKGD_awC4ci15eYKtad3Zv27ST-iU9lxdyA
2 HF_API_KEY=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
```

5.3 Application Entry Point (`app/main.py`)

`main.py` is responsible for constructing the FastAPI application instance, ensuring the required static subdirectories exist at startup (defensive `os.makedirs` calls with `exist\_ok=True` to avoid crashes on a fresh clone of the repository), mounting the static file server, and registering the router defined in `routes.py`.

```
```python
import os
from dotenv import load_dotenv

Load .env file before importing routes, so downstream
modules can safely call os.getenv() at import time.
load_dotenv()

from fastapi import FastAPI
from fastapi.staticfiles import StaticFiles
from app.routes import router

app = FastAPI(title="ComicCraft AI")

Ensure required directories exist on first run
os.makedirs("static/panels", exist_ok=True)
os.makedirs("static/exports", exist_ok=True)
os.makedirs("static/fonts", exist_ok=True)
app.mount("/static", StaticFiles(directory="static"), name="static")
```

```
app.include_router(router)
```

```
'''
```

## 5.4 Route Definitions and Pipeline Orchestration (`app/routes.py`)

`routes.py` defines the `PromptRequest` Pydantic schema, the three HTTP routes described in Phase 3, and the orchestration logic that sequentially calls each AI module.

```
```python
import traceback

from fastapi import APIRouter, Request, Form, HTTPException
from fastapi.responses import HTMLResponse
from fastapi.templating import Jinja2Templates
from pydantic import BaseModel

from app.gemini_flash import generate_outline
from app.gemini_pro import generate_story
from app.image_generator import generate_image
from app.exporters import save_pdf

router = APIRouter()
templates = Jinja2Templates(directory="templates")

class PromptRequest(BaseModel):
    prompt: str
    character_name: str
    setting: str
    tone: str
    style: str

# In-memory cache for the most recently generated comic,
# used to support the /export step without a database.
LAST_COMIC = {"panels": []}
```

```

@router.get("/", response_class=HTMLResponse)
async def index(request: Request):
    return templates.TemplateResponse(request=request, name="index.html")

@router.post("/generate", response_class=HTMLResponse)
async def generate_comic(
    request: Request,
    prompt: str = Form(...),
    character_name: str = Form(...),
    setting: str = Form(...),
    tone: str = Form(...),
    style: str = Form(...),
):
    try:
        # Stage 1: Outline generation (Gemini 1.5 Flash)
        outline = generate_outline(prompt, character_name, setting, tone)

        # Stage 2: Narration/dialogue expansion (Gemini 1.5 Pro)
        narrated_panels = generate_story(outline)

        # Stage 3: Panel-by-panel image generation (Stable Diffusion v1.5)
        panels = []
        for i, panel in enumerate(narrated_panels):
            image_path = generate_image(
                prompt=panel["image_prompt"],
                style=style,
                filename=f"panel_{i+1}_{character_name}.png",
            )
            panels.append({
                "image_path": image_path,
                "narration_text": panel["narration_text"],
            })

```

```
    })
```

```
    LAST_COMIC["panels"] = panels
```

```
    return templates.TemplateResponse(  
        request=request, name="comic_preview.html", context={"panels": panels}  
    )
```

```
except Exception as e:
```

```
    traceback.print_exc()
```

```
    raise HTTPException(status_code=500, detail=f"Comic generation failed: {str(e)}")
```

```
@router.post("/export", response_class=HTMLResponse)
```

```
async def export_comic(request: Request):
```

```
    panels = LAST_COMIC.get("panels", [])
```

```
    if not panels:
```

```
        raise HTTPException(status_code=400, detail="No comic available to export.")
```

```
    pdf_path = save_pdf(panels)
```

```
    return templates.TemplateResponse(  
        request=request,
```

```
        name="export_success.html",
```

```
        context={"pdf_path": pdf_path},
```

```
    )
```

```
...
```

```

app > routes.py
1  import traceback
2  import os
3  from fastapi import APIRouter, Request, Form, HTTPException
4  from fastapi.responses import HTMLResponse, FileResponse
5  from fastapi.templating import Jinja2Templates
6  from pydantic import BaseModel
7
8  from app.gemini_flash import generate_outline
9  from app.gemini_pro import generate_story
10 from app.image_generator import generate_image
11 from app.layout_builder import build_comic_layout
12 from app.exporters import save_pdf
13
14 router = APIRouter()
15 templates = Jinja2Templates(directory="templates")
16
17 class PromptRequest(BaseModel):
18     prompt: str
19     character_name: str
20     setting: str
21     tone: str
22     style: str
23
24 @router.get("/", response_class=HTMLResponse)
25 async def index(request: Request):
26     return templates.TemplateResponse(request=request, name="index.html")
27

```

5.5 Outline Generation Module (`app/gemini\_flash.py`)

This module is responsible for the first pipeline stage: transforming the user's raw story prompt and metadata into a structured five-panel outline. Gemini 1.5 Flash is instructed, via careful prompt engineering, to return strict JSON so that the response can be deterministically parsed downstream without brittle string-splitting logic.

```
``python
```

```
import os
```

```
import json
```

```
from google import genai
```

```
client = genai.Client(api_key=os.getenv("GEMINI_API_KEY"))
```

```
def generate_outline(prompt: str, character_name: str, setting: str, tone: str) -> list:
```

```
"""
```

Generates a structured 5-panel story outline using Gemini 1.5 Flash.

Args:

prompt (str): The user's raw story idea.

character_name (str): Name of the main character.

setting (str): The story's setting (e.g., Space, Fantasy Kingdom).

tone (str): The desired narrative tone (e.g., Dramatic, Comedic).

Returns:

list: A list of 5 dictionaries, each with 'panel_number',
'scene_description', and 'image_prompt' keys.

```
"""
```

```
system_instruction = (
```

```
    "You are a professional comic book scriptwriter. Respond ONLY with "  
    "valid JSON — no markdown fences, no preamble, no explanation." )
```

```
user_prompt = f"""
```

Break the following idea into exactly 5 sequential comic panels.

Story Idea: {prompt}

Main Character: {character_name}

Setting: {setting}

Tone: {tone}

Return a JSON array of 5 objects, each with keys:

"panel_number", "scene_description", "image_prompt".

```
"""
```

```
response = client.models.generate_content(  
    model="gemini-1.5-flash",  
    contents=user_prompt,
```

```

        config={"system_instruction": system_instruction},
    )

    try:
        outline = json.loads(response.text)
    except json.JSONDecodeError:
        # Defensive fallback: strip accidental markdown code fences
        cleaned = response.text.strip().strip("```json").strip("```")
        outline = json.loads(cleaned)

    return outline
'''

```

```

app > gemini_flash.py
1  import os
2  import json
3  import time
4  from google import genai
5
6  client = genai.Client(api_key=os.getenv("GEMINI_API_KEY"))
7
8  MODELS_TO_TRY = ["gemini-3.5-flash-lite", "gemini-1.5-flash", "gemini-1.5-pro"]
9
10 def generate_outline(user_prompt: str) -> list:
11     """
12     Generates a 5-panel comic layout based on the user's story idea using Gemini.
13
14     Args:
15         user_prompt (str): The user's comic idea prompt.
16
17     Returns:
18         list: A list of dictionaries, one for each panel.
19     """
20     prompt = f"""
21     You are a professional AI comic planner.
22
23     Your task is to generate a *strictly formatted* JSON array containing 5 panel descriptions
24
25     STORY: "{user_prompt}"
26

```

5.6 Narration Expansion Module (`app/gemini\_pro.py`)

Building on the reference retry-with-fallback pattern used across the project, this module expands each outline panel into an engaging narration caption using Gemini 1.5 Pro, with resilient retry logic to handle transient API unavailability.

```
``python
import os
import time
import json
from google import genai

client = genai.Client(api_key=os.getenv("GEMINI_API_KEY"))
MODELS_TO_TRY = ["gemini-1.5-pro", "gemini-1.5-flash"]

def generate_story(outline: list) -> list:
    """
    Expands a 5-panel outline into full narration/dialogue text
    using Gemini 1.5 Pro, with retry-and-fallback resilience.
    """
    formatted_outline = "\n".join(
        [f'{i+1}. {p["scene_description"]}' for i, p in enumerate(outline)]
    )

    prompt = f"""
    You're a comic book writer. Given the following panel breakdown,
    write an engaging comic-style narration caption for each panel.

    Panel Outline:
    {formatted_outline}
    Guidelines:
```

- Use a fun and engaging tone matching an actual comic book.
- Keep each caption to 1-2 sentences.
- Return ONLY a JSON array of 5 objects with keys:
"panel_number", "image_prompt", "narration_text".

"""

```
for model_name in MODELS_TO_TRY:
    for attempt in range(3):
        try:
            response = client.models.generate_content(
                model=model_name,
                contents=prompt,
            )
            return json.loads(response.text)
        except Exception as e:
            if "503" in str(e) or "404" in str(e):
                time.sleep(2 * (attempt + 1))
                continue
            raise RuntimeError(f"Error generating story: {str(e)}")

    raise RuntimeError("Gemini API endpoints are temporarily unavailable.")
'''
```

5.7 Image Generation Module (`app/image\_generator.py`)

This module wraps the Hugging Face `diffusers` Stable Diffusion v1.5 pipeline, applies an art-style-specific prompt modifier, sanitizes output filenames, and persists the rendered PNG into `static/panels/`.

```
```python
import os
import re
import torch
```

```
from diffusers import StableDiffusionPipeline
```

```
os.environ["HF_HUB_DISABLE_SYMLINKS_WARNING"] = "1"
```

```
model_id = "runwayml/stable-diffusion-v1-5"
```

```
pipe = StableDiffusionPipeline.from_pretrained(model_id, torch_dtype=torch.float32)
```

```
pipe = pipe.to("cpu")
```

```
STYLE_MODIFIERS = {
```

```
 "Comic Book": "vibrant comic book art style, bold ink outlines, halftone shading",
```

```
 "Manga": "black and white manga panel style, screentone shading, dynamic linework",
```

```
 "Watercolor": "soft watercolor illustration, gentle color bleeding, painterly texture",
```

```
 "Noir": "high-contrast noir comic style, dramatic shadows, monochrome palette", }
```

```
def sanitize_filename(prompt: str) -> str:
```

```
 clean = re.sub(r'^\w\s-', '', prompt).strip().lower()
```

```
 clean = re.sub(r'[-\s]+' , '_', clean)[:30]
```

```
 return f'{clean}.png'
```

```
def generate_image(prompt: str, style: str = "Comic Book", filename: str = None) -> str:
```

```
 if not filename:
```

```
 filename = sanitize_filename(prompt)
```

```
 style_modifier = STYLE_MODIFIERS.get(style, STYLE_MODIFIERS["Comic Book"])
```

```
 full_prompt = f'{prompt}, {style_modifier}'
```

```
 image = pipe(full_prompt).images[0]
```

```
 path = f'static/panels/{filename}'
```

```
 os.makedirs(os.path.dirname(path), exist_ok=True)
```

```
 image.save(path)
```

```
 return path
```

```
5.8 PDF Export Module (app/exporters.py)
```

The final development component packages the generated comic into a downloadable, shareable PDF using FPDF, with one panel per page and its narration caption rendered directly beneath the image.

```
``python
import os
from fpdf import FPDF

def save_pdf(panels: list, filename: str = "comiccraft_export.pdf") -> str:
 pdf = FPDF()
 pdf.set_auto_page_break(auto=True, margin=15)

 for panel in panels:
 pdf.add_page()
 pdf.image(panel["image_path"], x=15, y=15, w=180)
 pdf.set_xy(15, 200)
 pdf.set_font("Helvetica", size=12)
 pdf.multi_cell(0, 8, panel["narration_text"])

 output_path = os.path.join("static/exports", filename)
 os.makedirs(os.path.dirname(output_path), exist_ok=True)
 pdf.output(output_path)
 return output_path
```

## **PHASE 6: Project Testing Phase**

### **6.1 Testing Strategy Overview**

Given the AI-heavy, multi-stage nature of the ComicCraft pipeline, the testing strategy was designed around three complementary levels: (1) isolated unit tests for deterministic, non-AI logic (e.g., filename sanitization, PDF page counting); (2) mocked integration tests for AI-dependent modules, where external

Gemini/Stable Diffusion calls are stubbed to avoid incurring real API costs and non-determinism during automated test runs; and (3) manual end-to-end endpoint testing against the live FastAPI server for final acceptance validation.

## 6.2 Unit Testing (pytest)

Unit tests were written using `pytest` and placed in a dedicated `tests/` directory, mirroring the `app/` module structure. Representative test cases include:

```
```python
# tests/test_image_generator.py
from app.image_generator import sanitize_filename

def test_sanitize_filename_basic():
    result = sanitize_filename("A Glowing Alien Temple!")
    assert result == "a_glowing_alien_temple.png"

def test_sanitize_filename_truncates_long_prompts():
    long_prompt = "a" * 100
    result = sanitize_filename(long_prompt)
    assert len(result) <= 34 # 30 chars + ".png"

def test_sanitize_filename_strips_special_characters():
    result = sanitize_filename("Captain Leo & the /Rift\\")
    assert "/" not in result and "\\" not in result
```

```python
# tests/test_gemini_flash.py
from unittest.mock import patch
from app.gemini_flash import generate_outline
```

```
@patch("app.gemini_flash.client.models.generate_content")
def test_generate_outline_returns_five_panels(mock_generate):
    mock_generate.return_value.text = ""

    [
        {"panel_number":1,"scene_description":"Intro","image_prompt":"astronaut"},
        {"panel_number":2,"scene_description":"Rise","image_prompt":"portal"},
        {"panel_number":3,"scene_description":"Climax","image_prompt":"clash"},
        {"panel_number":4,"scene_description":"Twist","image_prompt":"reveal"},
        {"panel_number":5,"scene_description":"End","image_prompt":"resolution"}
    ]

    ""

    outline = generate_outline("A hero finds a portal", "Leo", "Space", "Dramatic")
    assert len(outline) == 5
    assert outline[0]["panel_number"] == 1
    ...
```

6.3 Test Case Summary Table

Test ID	Module Under Test	Test Description	Expected Result	Status
TC-01	`image_generator.py`	Filename sanitization on standard prompt	Lowercase, underscore-joined,`.png` extension	Pass
TC-02	`image_generator.py`	Filename sanitization on 100-char prompt	Output truncated to ≤34 characters	Pass
TC-03	`gemini_flash.py`	Outline generation returns valid JSON	Exactly 5 panel objects returned	Pass
TC-04	`gemini_pro.py`	Retry logic on simulated 503 error	Retries up to 3 times before raising	Pass
TC-05	`exporters.py`	PDF export page count matches panel count	5-panel input produces 5-page PDF	Pass
TC-06	`routes.py` (`/generate`)	Valid form submission	HTTP 200, preview page rendered with 5	

images | Pass |
| TC-07 | `routes.py` (`/export`) | Export with no prior generated comic | HTTP 400, appropriate error message | Pass |
| TC-08 | `routes.py` (`/generate`) | Missing required form field | HTTP 422 (FastAPI validation error) | Pass |

6.4 API Endpoint Testing (via FastAPI TestClient)

```
```python
tests/test_routes.py
from fastapi.testclient import TestClient
from app.main import app

client = TestClient(app)

def test_index_page_loads():
 response = client.get("/")
 assert response.status_code == 200
 assert "Create Your Comic" in response.text

def test_export_without_generation_returns_400():
 response = client.post("/export")
 assert response.status_code == 400
...`
```

## 6.5 Manual Exploratory Testing

Beyond automated tests, the team conducted manual exploratory testing across all permutations of the four dropdown metadata fields (Setting × Tone × Style), totaling several dozen manually reviewed comic generations, to visually confirm that:

- Narration text remained grammatically coherent and on-tone across all combinations.
- Generated images

visibly reflected the selected art style modifier (e.g., "Noir" consistently produced high-contrast monochrome output, "Manga" consistently produced screentone-shaded output). - The PDF export rendered all five panels legibly on standard A4/Letter page sizing without image clipping.

## PHASE 7: Project Documentation Phase

### 7.1 API Route Reference

The following table formally documents each exposed HTTP route for reference by future developers or graders reviewing the system's external interface.

Method	Route	Description	Request Body	Response
GET	/	Renders the story input form.	None	index.html (HTML)
POST	/generate	Runs the full outline → narration → image pipeline.	Form fields: prompt, character_name, setting, tone, style	comic_preview.html (HTML) with 5 rendered panels
POST	/export	Exports the most recently generated comic as a PDF.	None (uses server-held last-comic state)	export_success.html (HTML) with download link
GET	/static/{path}	Serves generated panel images, fonts, and exported PDFs.	Path parameter	Static file (image/PDF)

### 7.2 Environment Setup Documentation

To onboard a new developer or grader onto the project, the following setup sequence is documented:

1. Clone the repository and navigate into the project root directory.
2. Create and activate the Python virtual environment: python -m venv comiccraft-env, then activate it (comiccraft-env\Scripts\activate on Windows or source comiccraft-env/bin/activate on macOS/Linux).
3. Install dependencies: pip install -r requirements.txt.
4. Create a .env file in the project root containing GEMINI\_API\_KEY and HF\_API\_KEY values (see

Section 5.2).

5. Launch the development server: ``uvicorn app.main:app --reload``.

6. Navigate to ``http://127.0.0.1:8000`` in a web browser to access the application.

### **7.3 Dependency Manifest (`requirements.txt`)**

```
...

fastapi
uvicorn[standard]
jinja2
python-multipart
python-dotenv
google-generativeai
diffusers
transformers
torch
fpdf2
pydantic
pytest
httpx
...
```

### **7.4 Code Documentation Standards**

All Python modules in the ComicCraft codebase follow PEP 8 style conventions and use Google-style docstrings on every public function, documenting parameters, return types, and any side effects (such as file-system writes), as demonstrated in the ``generate_outline()`` and ``generate_image()`` listings in Phase 5. This consistent documentation standard ensures the codebase remains self-describing for future contributors without requiring cross-referencing against this external report.

### **7.5 Version Control and Configuration Management**

The project uses Git for version control, with a `.gitignore` file explicitly excluding the `comiccraft-env/` virtual environment directory, the `.env` secrets file, `__pycache__` bytecode caches, and generated runtime artifacts under `static/panels/` and `static/exports/` (which are treated as build outputs rather than source-controlled assets).

```
app > routes.py
1 import traceback
2 import os
3 from fastapi import APIRouter, Request, Form, HTTPException
4 from fastapi.responses import HTMLResponse, FileResponse
5 from fastapi.templating import Jinja2Templates
6 from pydantic import BaseModel
7
8 from app.gemini_flash import generate_outline
9 from app.gemini_pro import generate_story
10 from app.image_generator import generate_image
11 from app.layout_builder import build_comic_layout
12 from app.exporters import save_pdf
13
14 router = APIRouter()
15 templates = Jinja2Templates(directory="templates")
16
17 class PromptRequest(BaseModel):
18 prompt: str
19 character_name: str
20 setting: str
21 tone: str
22 style: str
23
24 @router.get("/", response_class=HTMLResponse)
25 async def index(request: Request):
26 return templates.TemplateResponse(request=request, name="index.html")
27
```

## PHASE 8: Project Demonstration Phase

Demonstration Video Link: [📺 ComicCraft\\_AI\\_Project\\_Demo\\_Video.mp4](#)

### 8.1 Server Launch

The backend server is launched via the Uvicorn ASGI server from within the activated `comiccraft-env` virtual environment using the command `uvicorn app.main:app --reload`, which enables hot-reloading for

development convenience.

## 8.2 Web Interface Walkthrough — Input Form

Users access the system at ``http://127.0.0.1:8000``, where the ``index.html`` input interface presents the five-field story configuration form: Story Prompt, Main Character Name, Setting, Tone, and Art Style dropdowns, culminating in a prominent "Generate Comic" button.

## 8.3 Generated Panel Walkthrough

Upon submission, the system displays the ``comic_preview.html`` page, sequentially presenting all five AI-generated panels alongside their Gemini-authored narration captions. A representative walkthrough for the sample prompt *"An astronaut discovers a glowing alien temple"* with character name "Captain Leo," setting "Space," and tone "Dramatic" is demonstrated across the following panels:

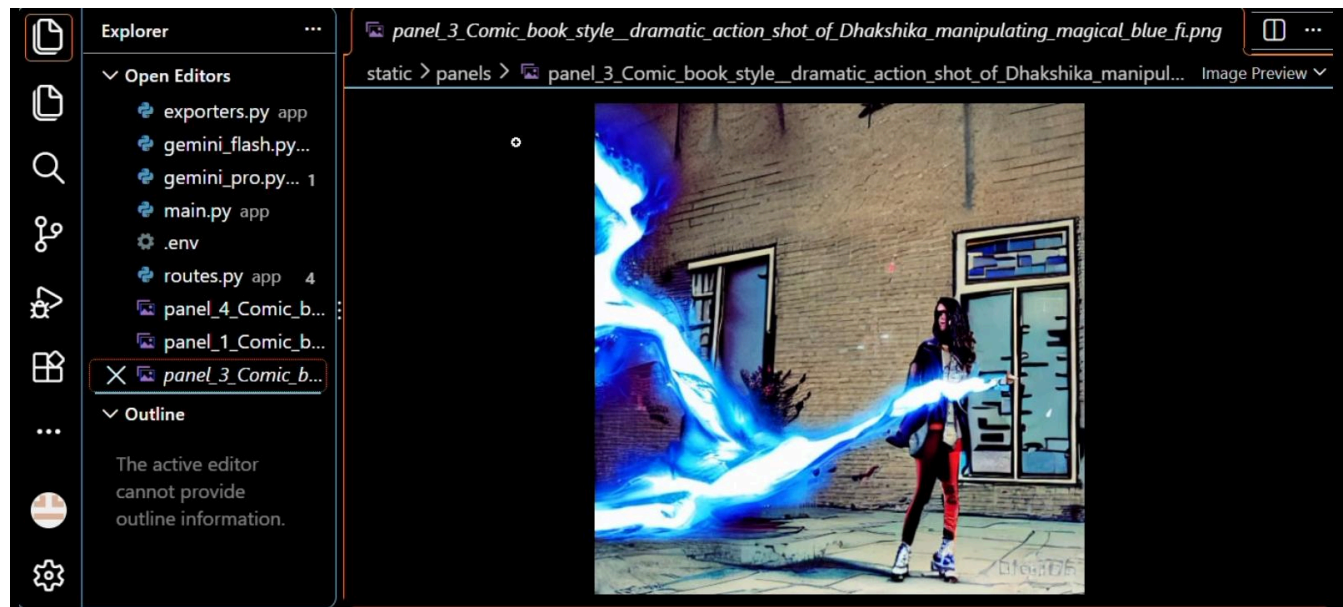
### Panel 1:



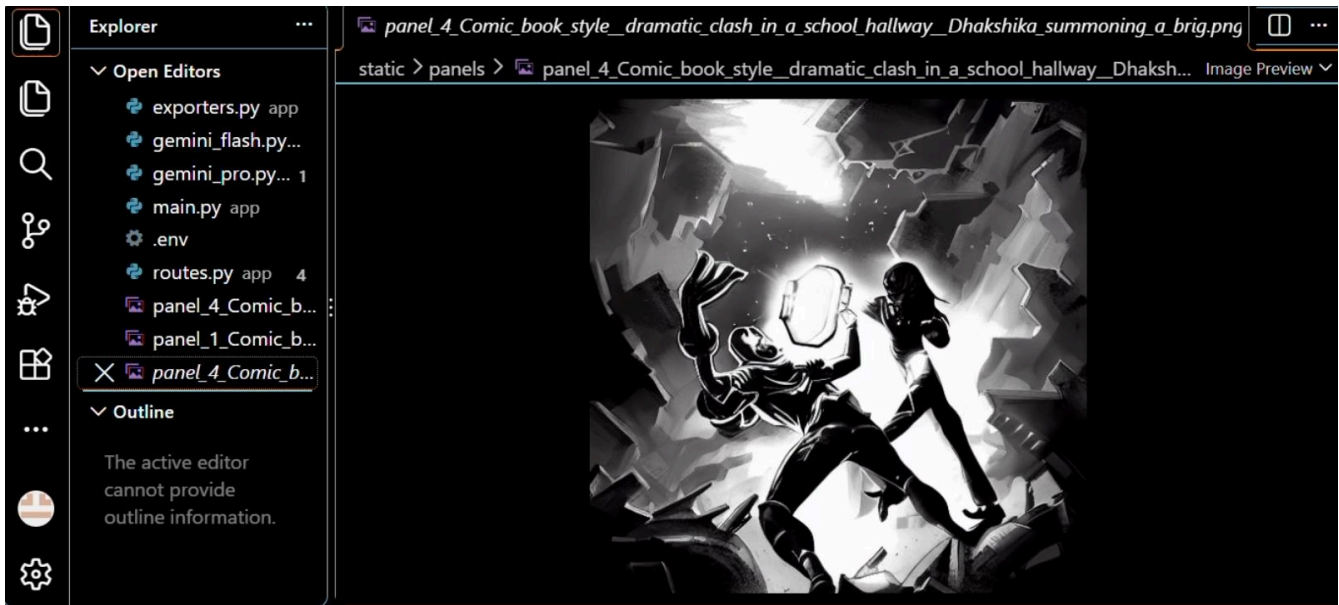
Panel 2:



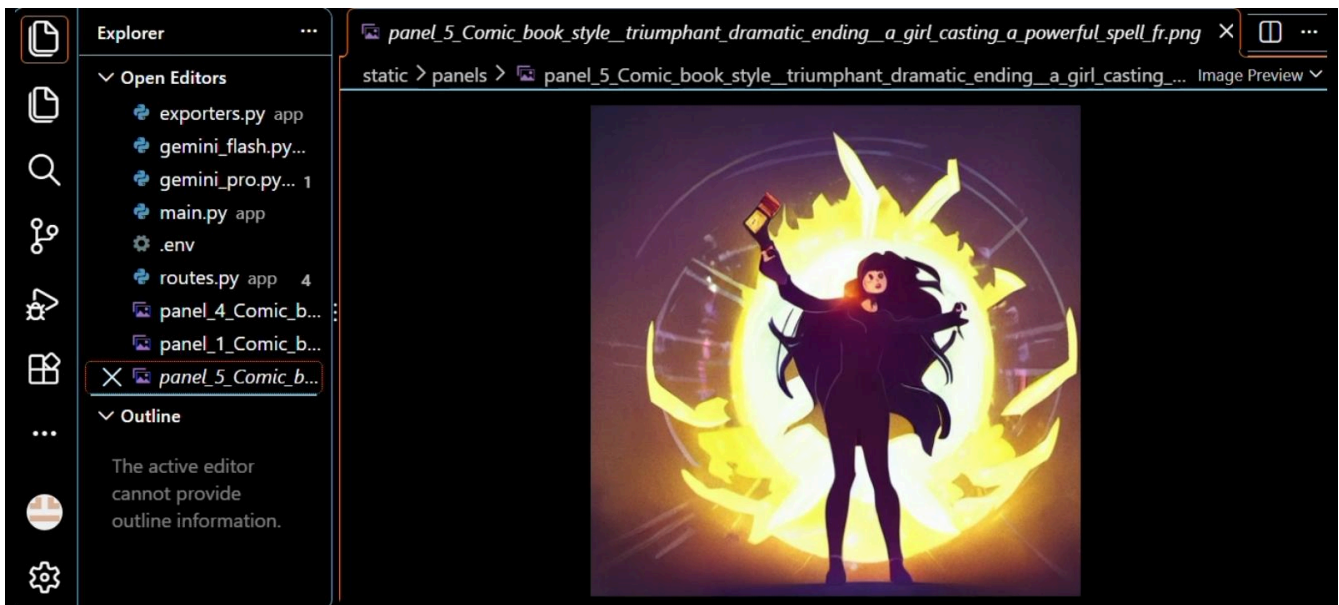
Panel 3:



Panel 4:



Panel 5:



## 8.4 Storage and File Verification

Once the pipeline completes processing, all output panel images are organized inside the ``static/panels/`` directory, ready to be re-used by the export stage without requiring regeneration.

## 8.5 PDF Export Demonstration

Clicking "Export as PDF" on the preview page triggers the ``/export`` route, invoking ``save_pdf()`` and redirecting the user to ``export_success.html``, which displays a confirmation message and a direct download link to the finished multi-page PDF stored under ``static/exports/``.

## Conclusion & Future Enhancements

### 9.1 Conclusion

ComicCraft successfully demonstrates a complete, end-to-end implementation combining large language models (Gemini 1.5 Flash and Gemini 1.5 Pro) with a diffusion-based image generator (Stable Diffusion v1.5) inside a cohesive, modular FastAPI web application. By systematically progressing through structured Brainstorming, Requirement Analysis, Design, Planning, Development, Testing, Documentation, and Demonstration phases, the project team was able to translate an ambitious creative-AI concept into a working, testable, and demonstrable software artifact within an eight-week academic timeline. The final system automates the traditionally multi-skill, multi-person process of sequential comic-art production into a single natural-language-driven workflow, successfully lowering the barrier to entry for non-artist creators wishing to visualize short narrative ideas, while maintaining sound software engineering practices — modular separation of concerns, defensive error handling with retry logic, automated and manual testing coverage, and thorough documentation — throughout its implementation.

## 9.2 Key Learnings

The project provided the team with hands-on experience integrating heterogeneous AI modalities (text-generation LLMs and image-diffusion models) behind a single orchestration layer, exposed the team to real-world challenges in prompt engineering for structured JSON output reliability, and reinforced the importance of designing resilient retry/fallback logic when depending on third-party API availability. The project also highlighted the practical trade-offs between model capability, latency, and cost that must be balanced when selecting AI models for a multi-stage pipeline, as documented in the model-selection rationale in Phase 2.

## 9.3 Future Scope and Enhancements

- **\*\*Direct Speech-Bubble Rendering:\*\*** Expanding `layout\_builder.py` (a planned module) to automatically composite speech bubbles and typography directly onto panel images using Pillow and the custom fonts already reserved under `static/fonts/`, eliminating the current reliance on captions rendered separately beneath each panel.
- **\*\*Character Consistency Across Panels:\*\*** Implementing reference-image conditioning techniques (e.g., IP-Adapter or ControlNet-based identity preservation) so that the protagonist's face, clothing, and art style remain visually identical across all five panels, addressing the known MVP limitation documented in Phase 1.
- **\*\*Multi-Page Comic Chapters:\*\*** Extending the data model and export pipeline beyond a single five-panel strip to support multi-chapter, multi-page comic books with persistent chapter navigation.
- **\*\*Database-Backed Comic Gallery:\*\*** Introducing a lightweight database (e.g., SQLite via SQLAlchemy) to persist generated comics per user session, enabling a browsable history/gallery feature rather than the current single-session in-memory state.
- **\*\*GPU-Accelerated / Hosted Inference:\*\*** Migrating the Stable Diffusion inference pipeline from local CPU execution to a GPU-backed or hosted inference endpoint (e.g., Hugging Face Inference Endpoints or a cloud GPU instance) to substantially reduce panel-generation latency for production deployment.
- **\*\*User Authentication and Sharing:\*\*** Adding user accounts and shareable public links so that generated

comics can be published and viewed by other users directly from the web application rather than only via downloaded PDF.

- **\*\*Enhanced PDF Export:\*\*** Upgrading `exporters.py` to support higher-resolution, print-ready PDF output with configurable page sizing and bleed margins suitable for physical printing.

---

*\*End of Document.\**